

ДНІПРОВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ОЛЕСЯ ГОНЧАРА

Факультет прикладної математики та інформаційних технологій

Кафедра комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №2

з дисципліни «Об'єктно орієнтоване програмування»

на тему «Наслідування і Поліморфізм»

Варіант 4

Виконавець:

студентка гр. ПС-24-1

Гарт Анна

Викладач:

доц. П. А. Дзюба

Дніпро

2026

Зміст

<i>Постановка задачі</i>	3
<i>Індивідуальне завдання</i>	3
<i>Опис розв'язку</i>	4
<i>Вихідний текст програми розв'язку задачі</i>	5
Файл <i>Lib.h</i>	5
Файл <i>Lab2.cpp</i>	8
<i>Опис інтерфейсу програми</i>	13

Постановка задачі. Скласти об'єктно-орієнтовану програму на мові C++, яка в діалоговому режимі керує графічними об'єктами, що відображаються на екрані дисплею.

По натисненню клавіші F1 програма виводить на екран коротку підказку по усіх наявних командах/клавішах; наприклад: як створити об'єкт, як зрушити з місця, як перейти до «наступного» об'єкту тощо.

Програма повинна підтримувати такі загальні елементи поведінки графічних об'єктів:

1. Створення довільної кількості графічних об'єктів із фіксованої множини їх різновидів.
2. Активізація/візуалізація графічного об'єкту за вибором користувача.
3. Виконання операцій над поточним (активним) об'єктом:
 - a. переміщення зі слідом/без;
 - b. збереження і відновлення стану об'єкту;
 - c. зміна кольору;
 - d. зміна розміру;
 - e. зміна стану видимий/невидимий.
4. Агрегація, тобто утворення нових об'єктів з вже створених. При агрегації об'єктів забезпечити наступні можливості:
 - a. агрегація агрегатів;
 - b. можливість виконувати над агрегатами ті ж дії, що і над «звичайними» об'єктами.

Програма повинна надати користувачеві можливість зберігати поточну конфігурацію програми у вказаний користувачем текстовий файл на диску і завантажувати поточну конфігурацію програми із вказаного користувачем текстового файлу на диску.

Індивідуальне завдання. Поставимо у відповідність i -му пункту завдання номер $N = 4$ в двійковій СЧ. Номера підпунктів завдання будемо вважати коефіцієнтами розкладення 4 за степенями двійки. Таким чином, номеру $N = 4$ відповідає номер індивідуального завдання 00100, тобто пункти 1.0, 2.0, 3.1, 4.0, 5.0.

Отже, варіант поведінки об'єктів:

- 1.0. Ім'я конфігураційного файлу задане у командному рядку при запуску програми.
- 2.0. Рух об'єкта в автоматичному режимі по заданому закону.
- 3.1. Зміна кольору об'єкта під впливом іншого об'єкта.
- 4.0. Деформація об'єкта по команді.

5.0. Збирання/запам'ятовування агрегованого образу відбувається дублюванням об'єктів, з яких будується агрегат.

Опис розв'язку. У результаті виконання лабораторної роботи було розроблено WinAPI застосунок з використанням бібліотеки GDI+, який моделює рух графічних об'єктів у двовимірному просторі. У програмі реалізовано три типи об'єктів: кола, прямокутники та трикутники, які рухаються по екрану з заданою швидкістю, відбиваються від меж вікна та взаємодіють між собою при зіткненнях.

Основою програми є абстрактний базовий клас `GraphicObject`, який містить спільні властивості для всіх об'єктів, такі як координати положення, швидкість руху, розміри та колір. У цьому класі реалізовано основну логіку руху об'єктів, а саме метод `update`, який оновлює координати відповідно до швидкості та забезпечує відскок від меж вікна. Також реалізовано метод перевірки зіткнення `collide` на основі AABB-алгоритму, який визначає перетин прямокутних областей об'єктів. У випадку зіткнення викликається метод `onCollision`, який змінює колір об'єктів та моделює їх відштовхування один від одного шляхом зміщення координат і зміни напрямку руху.

На основі базового класу реалізовано три похідні класи: `RectangleObj`, `CircleObj` та `TriangleObj`. Кожен з них перевизначає метод `draw`, який відповідає за відображення відповідної фігури за допомогою засобів GDI+. Прямокутники малюються через `FillRectangle`, кола через `FillEllipse`, а трикутники — через `FillPolygon` із заданими вершинами.

У програмі реалізовано динамічне оновлення сцени за допомогою таймера WinAPI, який періодично викликає оновлення позицій об'єктів та перевірку зіткнень між ними. При кожному циклі виконання перевіряються всі пари об'єктів, і у разі їх перетину застосовується реакція зіткнення. Відображення сцени виконується у функції `WM_PAINT`, де очищається екран і заново малюються всі об'єкти в їх актуальних координатах.

Також реалізовано взаємодію користувача з програмою. При натисканні лівої кнопки миші у точці курсора створюється випадковий об'єкт одного з трьох типів з випадковими параметрами руху та розміру. При натисканні правої кнопки миші відбувається видалення об'єкта, який знаходиться під курсором. За допомогою клавіш 1, 2 та 3 можна створювати відповідно коло, трикутник або прямокутник у випадкових координатах. Клавіша F1 виводить довідкове повідомлення з керуванням програмою. Крім того, за допомогою колеса миші можна змінювати швидкість вибраного об'єкта.

На початку роботи програми ініціалізується GDI+, створюється головне вікно та запускається таймер. Початкові об'єкти створюються програмно у функції `InitObjects`, де задаються їх координати, розміри та швидкості руху.

Вихідний текст програми розв'язку задачі.

Файл Lib.h

```
#ifndef LIB_H
#define LIB_H

#include <windows.h>
#include <gdiplus.h>
#include <vector>
#include <memory>
#include <cmath>

using namespace Gdiplus;
using namespace std;

#pragma comment(lib, "gdiplus.lib")

// ===== БАЗОВИЙ КЛАС =====
class GraphicObject {
public:
    float x, y;
    float vx, vy;
    float w, h;
    Color color;

    GraphicObject(float x, float y, float w, float h, float vx, float vy, Color c)
        : x(x), y(y), w(w), h(h), vx(vx), vy(vy), color(c) {}

    virtual void draw(Graphics& g) = 0;

    virtual void update(int W, int H) {
        x += vx;
        y += vy;

        // відскок від стін (контур)
        if (x < 0 || x + w > W) vx = -vx;
        if (y < 0 || y + h > H) vy = -vy;

        if (x < 0) x = 0;
        if (y < 0) y = 0;
        if (x + w > W) x = W - w;
        if (y + h > H) y = H - h;
    }

    // КОНТУРНЕ ЗІТКНЕННЯ (AABB)
    bool collide(GraphicObject* o) {
        return !(x + w < o->x || o->x + o->w < x ||
                y + h < o->y || o->y + o->h < y);
    }
}
```

```

// ВІДШТОВХУВАННЯ + ЗМІНА КОЛЬОРУ
void onCollision(GraphicObject* o) {

    // ===== зміна кольору =====
    color = Color(255,
        (color.GetR() + 80) % 255,
        (color.GetG() + 50) % 255,
        (color.GetB() + 120) % 255
    );

    o->color = Color(255,
        (o->color.GetR() + 90) % 255,
        (o->color.GetG() + 60) % 255,
        (o->color.GetB() + 100) % 255
    );

    // ===== центри =====
    float cx1 = x + w / 2;
    float cy1 = y + h / 2;

    float cx2 = o->x + o->w / 2;
    float cy2 = o->y + o->h / 2;

    float dx = cx1 - cx2;
    float dy = cy1 - cy2;

    float absX = fabs(dx);
    float absY = fabs(dy);

    // ===== ВІДШТОВХУВАННЯ ОБ'ЄКТІВ МІЖ СОБОЮ =====
    if (absX > absY) {

        // удар зліва/справа → як вертикальна стіна
        vx = -vx;

        // розштовхуємо
        float push = (w + o->w) / 2 - absX;

        if (dx > 0) {
            x += push / 2;
            o->x -= push / 2;
        } else {
            x -= push / 2;
            o->x += push / 2;
        }
    }
    else {

        // удар зверху/знизу → як горизонтальна стіна
        vy = -vy;

        float push = (h + o->h) / 2 - absY;
    }
}

```

```

        if (dy > 0) {
            y += push / 2;
            o->y -= push / 2;
        } else {
            y -= push / 2;
            o->y += push / 2;
        }
    }
}

virtual ~GraphicObject() {}
};

// ===== ПРЯМОКУТНИК =====
class RectangleObj : public GraphicObject {
public:
    RectangleObj(float x, float y, float w, float h, float vx, float vy)
        : GraphicObject(x, y, w, h, vx, vy, Color(255, 0, 0, 200)) {}

    void draw(Graphics& g) override {
        SolidBrush brush(color);
        g.FillRectangle(&brush, x, y, w, h);
    }
};

// ===== КОЛО =====
class CircleObj : public GraphicObject {
public:
    CircleObj(float x, float y, float r, float vx, float vy)
        : GraphicObject(x, y, r * 2, r * 2, vx, vy, Color(255, 0, 200, 0)) {}

    void draw(Graphics& g) override {
        SolidBrush brush(color);
        g.FillEllipse(&brush, x, y, w, h);
    }
};

// ===== ТРИКУТНИК =====
class TriangleObj : public GraphicObject {
public:
    TriangleObj(float x, float y, float size, float vx, float vy)
        : GraphicObject(x, y, size, size, vx, vy, Color(255, 200, 100, 50)) {}

    void draw(Graphics& g) override {

        PointF p1(x + w / 2, y);
        PointF p2(x, y + h);
        PointF p3(x + w, y + h);

        PointF points[3] = { p1, p2, p3 };
    }
};

```

```

        SolidBrush brush(color);
        g.FillPolygon(&brush, points, 3);
    }
};

#endif

```

Файл Lab2.cpp

```

#define UNICODE
#define _UNICODE

#include "Lib.h"
#include <fstream>

vector<shared_ptr<GraphicObject>> objects;
shared_ptr<GraphicObject> selected = nullptr;

ULONG_PTR token;

// ===== ІНІЦІАЛІЗАЦІЯ GDI =====
void InitGDI() {
    GdiplusStartupInput input;
    GdiplusStartup(&token, &input, nullptr);
}

void ShutdownGDI() {
    GdiplusShutdown(token);
}

// ===== ЗАВАНТАЖЕННЯ КОНФІГУРАЦІЇ =====
void InitObjects() {

    objects.push_back(make_shared<TriangleObj>(
        100, 100, 60, 4, 2
    ));

    objects.push_back(make_shared<TriangleObj>(
        300, 200, 80, -1, 3
    ));

    objects.push_back(make_shared<CircleObj>(
        100, 100, 40, 5, 4
    ));

    objects.push_back(make_shared<RectangleObj>(
        300, 200, 120, 80, -1, 2
    ));

    objects.push_back(make_shared<CircleObj>(
        500, 300, 30, 3, -2
    ));
}

```

```

    objects.push_back(make_shared<RectangleObj>(
        200, 400, 150, 60, 2, -1
    ));
}

// ===== ОБРОБНИК ВКЛАДКИ =====
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {

    switch (msg) {

    case WM_CREATE:
        SetTimer(hwnd, 1, 30, NULL);
        InitObjects();
        srand((unsigned)time(NULL));
        return 0;

    case WM_TIMER: {

        RECT rc;
        GetClientRect(hwnd, &rc);

        int W = rc.right;
        int H = rc.bottom;

        for (auto& o : objects)
            o->update(W, H);

        for (size_t i = 0; i < objects.size(); i++) {
            for (size_t j = i + 1; j < objects.size(); j++) {
                if (objects[i]->collide(objects[j].get())) {
                    objects[i]->onCollision(objects[j].get());
                    objects[j]->onCollision(objects[i].get());
                }
            }
        }

        InvalidateRect(hwnd, NULL, TRUE);
        return 0;
    }

    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hwnd, &ps);

        Graphics g(hdc);
        g.Clear(Color(255, 25, 25, 25));

        for (auto& o : objects)
            o->draw(g);

        EndPaint(hwnd, &ps);
        return 0;
    }
    }
}

```

```

}

// ===== ЛКМ — створення випадкового об'єкта =====
case WM_LBUTTONDOWN: {

    int x = LOWORD(lParam);
    int y = HIWORD(lParam);

    int type = rand() % 3;

    float vx = (rand() % 7 - 3);
    float vy = (rand() % 7 - 3);
    float size = 20 + rand() % 50;

    if (type == 0) {
        objects.push_back(make_shared<CircleObj>(x, y, size, vx, vy));
    }
    else if (type == 1) {
        objects.push_back(make_shared<RectangleObj>(x, y, size, size, vx, vy));
    }
    else {
        objects.push_back(make_shared<TriangleObj>(x, y, size, vx, vy));
    }

    return 0;
}

// ===== ПКМ — видалення об'єкта =====
case WM_RBUTTONDOWN: {

    int mx = LOWORD(lParam);
    int my = HIWORD(lParam);

    selected = nullptr;

    for (auto it = objects.begin(); it != objects.end(); ++it) {

        float x = (*it)->x;
        float y = (*it)->y;
        float w = (*it)->w;
        float h = (*it)->h;

        if (mx >= x && mx <= x + w &&
            my >= y && my <= y + h) {

            selected = *it;
            objects.erase(it);
            break;
        }
    }

    return 0;
}

```

```

}

// ===== 1 — коло =====
case WM_KEYDOWN:

    if (wParam == '1') {

        int x = rand() % 700;
        int y = rand() % 500;

        float vx = (rand() % 7 - 3);
        float vy = (rand() % 7 - 3);
        float r = 20 + rand() % 40;

        objects.push_back(make_shared<CircleObj>(x, y, r, vx, vy));
    }

// ===== 2 — трикутник =====
else if (wParam == '2') {

    int x = rand() % 700;
    int y = rand() % 500;

    float vx = (rand() % 7 - 3);
    float vy = (rand() % 7 - 3);

    float size = 30 + rand() % 50;

    objects.push_back(make_shared<TriangleObj>(x, y, size, vx, vy));
}

// ===== 3 — прямокутник =====
else if (wParam == '3') {

    int x = rand() % 700;
    int y = rand() % 500;

    float vx = (rand() % 7 - 3);
    float vy = (rand() % 7 - 3);

    float w = 30 + rand() % 60;
    float h = 30 + rand() % 60;

    objects.push_back(make_shared<RectangleObj>(x, y, w, h, vx, vy));
}

// ===== F1 ДОПОМОГА =====
else if (wParam == VK_F1) {

    MessageBoxW(hwnd,
        L"Керування:\n\n"
        L"ЛКМ - створити випадковий об'єкт\n"

```

```

        L"1 - коло\n"
        L"2 - трикутник\n"
        L"3 - прямокутник\n"
        L"ПКМ - видалити об'єкт\n",
        L"Довідка",
        MB_OK | MB_ICONINFORMATION
    );
}

return 0;

// ===== КОЛЕСО МИШІ =====
case WM_MOUSEWHEEL: {

    if (!selected) return 0;

    int delta = GET_WHEEL_DELTA_WPARAM(wParam);

    selected->vx += (delta > 0 ? 1 : -1);
    selected->vy += (delta > 0 ? 1 : -1);

    return 0;
}

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}

return DefWindowProc(hwnd, msg, wParam, lParam);
}

// ===== ГОЛОВНА ФУНКЦІЯ =====
int WINAPI wWinMain(HINSTANCE hInstance, HINSTANCE, PWSTR, int nCmdShow) {

    InitGDI();

    WNDCLASS wc = {};
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = L"LabWindow";

    RegisterClass(&wc);

    HWND hwnd = CreateWindow(
        L"LabWindow",
        L"GDI+ Lab OOP",
        WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, CW_USEDEFAULT,
        900, 600,
        nullptr, nullptr, hInstance, nullptr
    );
};

```

```

ShowWindow(hwnd, nCmdShow);

MSG msg = {};
while (GetMessage(&msg, nullptr, 0, 0)) {
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}

ShutdownGDI();
return 0;
}

```

Опис інтерфейсу програми. Інтерфейс програми реалізовано на основі стандартного віконного застосунку WinAPI з використанням бібліотеки GDI+. Головне вікно програми створюється функцією CreateWindow та має фіксовані розміри, у межах яких відбувається вся візуалізація графічних об'єктів. Фон сцени задається у процедурі WM_PAINT та очищається темним кольором, після чого на екран послідовно відображаються всі об'єкти з поточного списку.

Візуальна частина інтерфейсу складається з графічних фігур трьох типів: кіл, прямокутників і трикутників, які відображаються у вигляді заповнених кольорових фігур. Кожен об'єкт має власний колір, координати та розміри, а також рухається по екрану, що створює динамічну анімацію. При зіткненні об'єкти змінюють колір, що дозволяє користувачу візуально відслідковувати їх взаємодію.

Керування програмою здійснюється за допомогою миші та клавіатури. Ліва кнопка миші дозволяє створювати нові об'єкти у точці натискання з випадковими параметрами розміру та швидкості, що робить інтерфейс інтерактивним. Права кнопка миші використовується для видалення об'єкта, який знаходиться під курсором. Клавіші 1, 2 та 3 дозволяють швидко додавати відповідно коло, трикутник або прямокутник у випадкових координатах сцени.

Додатково реалізовано довідкове вікно, яке викликається клавішею F1 і містить інформацію про всі доступні елементи керування. За допомогою колеса миші користувач може змінювати швидкість вибраного об'єкта, що дозволяє впливати на його поведінку під час руху.

Таким чином, інтерфейс програми є мінімалістичним і побудований навколо графічної області відображення, при цьому основна взаємодія користувача здійснюється через мишу та клавіатуру без використання додаткових елементів управління (кнопок або панелей).